

LUCRARE CALCUL NUMERIC

BANESIU SEVER

Regula Trapezelor

Exercise. Estimati numarul de subintervale necesar pentru a calcula

$$\int_0^1 e^{-x^2} dx$$

cu 6 zecimale corecte (eroare absoluta $\leq \frac{1}{2} \times 10^{-6}$) cu *regula trapezelor*.

Proof. Fie $f(x) = e^{-x^2}$, pentru gasirea numarului de intervale necesar folosim formula pentru calculul erorii la estimarea valorii integralei definite atunci cand $f''(x)$ este continua pe intervalul $[a, b]$:

$$|eroare| \leq \frac{(b-a)^3}{12n^2} |f''(M)|$$

unde $|f''(M)|$ este maximul lui $|f''(x)|$ pentru $\forall x \in [a, b]$ si n este numarul de subintervale.

Calculam $f''(x)$ si obtinem:

$$f''(x) = e^{-x^2} (4x^2 - 2)$$

Pentru determinarea valorii absolute maxime a functiei $f''(x)$ vom folosi derivata sa adica:

$$f^{(3)}(x) = e^{-x^2} 4x(3 - 2x^2)$$

Valorile x pentru care $f^{(3)}(x) = 0$ sunt: 0 si $(\pm \frac{3}{2})^{\frac{1}{2}} \approx \pm 1.225$, se observa ca $f^{(3)}(x)$ este monotona pe intervalul $[0, 1]$. Calculam deci valoarea functiei $f^{(3)}(x)$ pe capetele intervalului si obtinem $f^{(3)}(0) = -2$ si $f^{(3)}(1) \approx 0.735$. Concluzionam:

$$\max(|f''(x)|, \forall x \in [0, 1]) = 2$$

Calculam numarul de intervale necesar inlocuind in formula pentru calculul erorii:

$$|eroare| \leq \frac{(1-0)^3}{12n^2} \times 2 = \frac{1}{2} \times 10^{-6}$$

rescriem $n = \frac{10^3}{\sqrt{3}}$ adica $n \approx 577.350$, dat fiind ca n reprezinta numarul de subintervale, deci $n \in \mathbb{N}$ rotunjim valoarea gasita mai devreme si spunem ca $n = 578$, adica sunt necesare cel putin 578 intervale pentru a estima valoarea $\int_0^1 f(x) dx$ cu 6 zecimale exacte folosind *regula trapezului*. $\square$

IMPLEMENTARE OCTAVE

In continuare prezint o implementare in Octave/Matlab care foloseste *regula trapezului* pentru calculul efectiv al aproximarii integralei definite:

$$\int_0^1 e^{-x^2} dx$$

Fisierul *fe.m* contine definitia functiei $f(x)$:

```
function R = fe(x)
    R = e^(-(x^2));
endfunction
```

Fisierul *aprox_trapez.m* defineste functia *aprox_trapez* care primeste 4 parametrii:

- (1) f functia a carei integrala definita pe domeniul $[a, b]$ urmeaza a fi aproximata
- (2) a inceputul intervalului
- (3) b sfarsitul intervalului
- (4) n numarul de subintervale folosit pentru aproximare

```
function R = aprox_trapez(f, a, b, n)
    inc = (b-a)/n;
    sum = 0.0;
    fa = f(a);
    for i = 0:n-1
        b = a + inc;
        fb = f(b);
        sum += fa + fb;
        a = b;
        fa = fb;
    endfor
    R = sum * 0.5 * inc;
endfunction
```

Example. Rularea in Octave a codului produce urmatorul rezultat:

```
octave:1> format long
octave:2> aprox_trapez(@fe, 0, 1, 578)
ans = 0.746823949285986
```

Formula repetata a lui Simpson

Exercise. Rezolvati problema precedenta folosind *Formula repetata a lui Simpson*.

Proof. Pastram notatia $f(x) = e^{-x^2}$, pentru gasirea numarului de intervale necesar folosim formula pentru calculul erorii la estimarea valorii integralei definite atunci cand $f^{(4)}(x)$ este continua pe intervalul $[a, b]$:

$$|eroare| \leq \frac{(b-a)^5}{180n^4} |f^{(4)}(M)|$$

unde $|f^{(4)}(M)|$ este maximul lui $|f^{(4)}(x)|$ pentru $\forall x \in [a, b]$ si n este numarul de subintervale.

Refolosind calculele anterioare, ajungem usor la urmatorul rezultat:

$$f^{(4)}(x) = 4e^{-x^2}(4x^4 - 12x^2 + 3)$$

Pentru determinarea valorii absolute maxime a functiei $f^{(4)}(x)$ vom folosi derivata sa adica:

$$f^{(5)}(x) = -8e^{-x^2}x(4x^4 - 20x^2 + 15)$$

Valorile x pentru care $f^{(5)}(x) = 0$ sunt: $0, \pm\sqrt{\frac{1}{2}(5 - \sqrt{10})} \approx \pm 0.958572$ si $\pm\sqrt{\frac{1}{2}(5 + \sqrt{10})} \approx \pm 2.02018$. Se observa ca unul din capetele domeniului coincide cu unul din punctele critice si unul se afla in interiorul domeniului. Calculam deci valorile functiei $f^{(4)}(x)$ in punctele: $0, \sqrt{\frac{1}{2}(5 - \sqrt{10})}$ si 1 si obtinem:

$$f^{(4)}(0) = 12$$

$$f^{(4)}(\sqrt{\frac{1}{2}(5 - \sqrt{10})}) = -16(-2 + \sqrt{10})e^{-\frac{5}{2} + \sqrt{\frac{5}{2}}} \approx -7.41948$$

$$f^{(4)}(1) = -\frac{20}{e} \approx -7.35758$$

Concluzionam:

$$\max(|f^{(4)}(x)|, \forall x \in [0, 1]) = 12$$

Calculam numarul de intervale necesar inlocuind in formula pentru calculul erorii:

$$|eroare| \leq \frac{(1-0)^5}{180n^4} \times 12 = \frac{1}{2} \times 10^{-6}$$

rescriem $n = \sqrt[4]{\frac{2 \times 10^6}{15}}$ adica $n \approx 19.108$, dat fiind ca n reprezinta numarul de subintervale, deci $n \in \mathbb{N}$ rotunjim valoarea gasita mai devreme si spunem ca $n = 20$, adica sunt necesare cel putin 20 intervale pentru a estima valoarea $\int_0^1 f(x)dx$ cu 6 zecimale exacte folosind *formula repetata a lui Simpson*. De asemenea, numarul de subintervale satisface conditia de a fi par. $\square$

IMPLEMENTARE OCTAVE

În continuare prezint o implementare în Octave/Matlab care folosește *formula repetată a lui Simpson* pentru calculul efectiv al aproximării integralei definite:

$$\int_0^1 e^{-x^2} dx$$

Fisierul *fe.m* conține definiția funcției $f(x)$ și rămâne neschimbat față de exemplul precedent.

Fisierul *aprox_simpson.m* definește funcția *aprox_simpson* care primește 4 parametri:

- (1) f funcția a cărei integrală definită pe domeniul $[a, b]$ urmează a fi aproximată
- (2) a începutul intervalului
- (3) b sfârșitul intervalului
- (4) n numărul de subintervale folosit pentru aproximare

```
function R = aprox_simpson(f, a, b, n)
    n += mod(n, 2); # Fortam n sa fie par
    inc = (b-a)/n;
    sum = f(a) + f(b);
    x = a + inc;
    m = 4;
    for i = 0:n-2
        sum += m * f(x);
        m = 6 - m; # m == 2 ? 4 : 2
        x += inc;
    endfor
    R = inc * sum / 3;
endfunction
```

Example. Rularea în Octave a codului produce următorul rezultat:

```
octave:1> format long
octave:2> aprox_simpson(@fe, 0, 1, 20)
ans = 0.746824183875915
```

INTERPOLAREA LAGRANGE

Exercise. Sa se calculeze $\sqrt{115}$ cu trei zecimale exacte folosind *interpolarea Lagrange*.

Proof. Algoritmul ales pentru rezolvarea acestei probleme este o combinatie intre *metoda bisectiei domeniului* si *interpolarea Lagrange*. Algoritmul functioneaza ca cel din *metoda bisectiei* modificarea survine in modul de lucru cu punctele de bisectie: acestea sunt plasate intr-un sir si sunt folosite pentru estimarea valorii radicalului. Din cauza interpolarii numarul de diviziuni este redus semnificativ (17 diviziuni in general sunt suficiente pentru a calcula radicalul unor numere de ordinul milioanei cu trei zecimale exacte). Cand intre doua estimari consecutive exista o eroare mai mica decat cea stabilita executia algoritmului se opreste. $\square$

IMPLEMENTARE OCTAVE

Fisierul *lagrange.m* contine functia *lagrange* care estimeaza valoarea intr-un punct pe baza unui set de date. Parametrii functiei sunt:

- (1) *x* o lista cu punctele in care valorile functiei se cunosc
- (2) *y* o lista cu valorile care se cunosc
- (3) *pos* punctul in care se doreste estimarea valorii

```
function R = lagrange(x, y, pos)
    assert (length(x) == length(y));
    R = 0;
    len = length(x);
    for i = 1:len
        weight = 1;
        for j = 1:len
            if(i!=j)

                weight *= (pos - x(j))/(x(i) - x(j));
            endif
        endfor
        R += weight * y(i);
    endfor
endfunction
```

Fisierul *lagrange_sqrt.m* contine functia *lagrange_sqrt* care estimeaza valoarea unui radical de ordin 2 pentru un numar. Parametrii functiei sunt:

- (1) *num* numarul a carui radacina se aproximeaza
- (2) *decimals* numarul de zecimale exacte dorite

```
function R = lagrange_sqrt(num, decimals)
    assert (num >= 1);
    decimals = abs(decimals);
    x = [1, num**2];
    y = [1, num];
    err = 0.5 * (10^-decimals);
    left = 0;
    right = num;
    R = lagrange(x, y, num);
    found = 0;
    while(!found)
        next_x = (left+right)/2;
        x(length(x)+1)=next_x ** 2;
        y(length(y)+1)=next_x;
        if(next_x ** 2 > num)
            right = next_x;
        else
            left = next_x;
        endif
        new_estimate = lagrange(x,y,num);
        if(abs(new_estimate - R) <= err)
            found = true;
        endif
        R = new_estimate;
    endwhile
endfunction
```

Example. Rularea in Octave a codului produce urmatorul rezultat:

```
octave:1> format long
octave:2> lagrange_sqrt(115, 3)
ans = 10.7238071094741
```